

Project 2: SDN-Based Stateless Firewall Project

Student Name: Mark Khusid

Email: mkhusid1@asu.edu

Submission Date: November 10, 2024

Class Name and Term: CSE548 Summer 2021

I. PROJECT OVERVIEW

The purpose of this project is to learn about Software Defined Networking. In this project an Open Virtual Switch is created that operates on the OpenFlow protocol. A POX instance acts as the controller for the Open Virtual Switch. Containers are instantiated using Mininet to create four containers. Layer 2 and Layer 3 firewall rules are applied to the controller using a firewall application running the application plane. Various tests are performed in order to test the functionality of the firewall application.

II. NETWORK SETUP

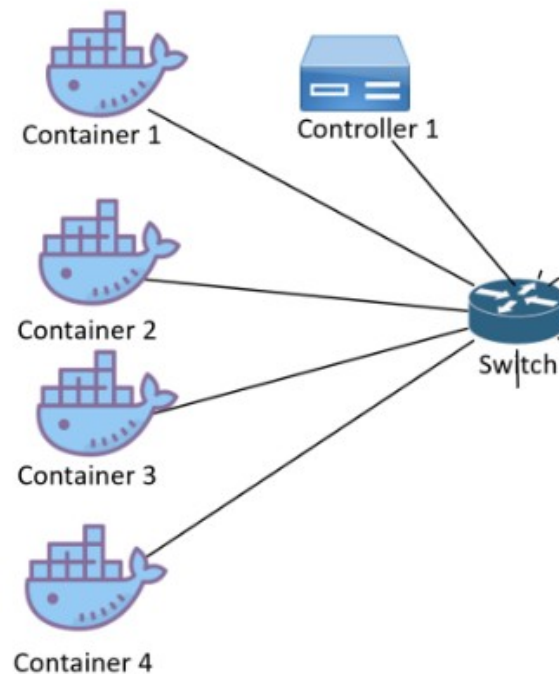


Figure CS-CNS-00101.3
Network topology for SDN.

(Figure was copied from project directions and modified)

The figure above shows the network topology. The default IP addresses were changed such that the containers have the following IP addresses:

Container 1: 192.168.2.10
Container 2: 192.168.2.20
Container 3: 192.168.2.30
Container 4: 192.168.2.40

The default firewall rules did not prevent reachability between the containers. Firewall rules were added per the Project instructions and tests were performed to check the functionality of the firewall with the new rules.

III. SOFTWARE

POX was used as the controller for the Open Virtual Switch. Mininet was used to set up containers connected to the switch.

IV. PROJECT DESCRIPTION

To set up the controller, the following POX command was used:

```
ubuntu@ubuntu:~/pox$ ./pox.py openflow.of_01 --port=6655 pox.forwarding.l2_learning pox.forwarding.L3Firewall --l2config="l2firewall.config" --l3config="l3firewall.config"
```

To set up the containers, the following Mininet command was used:

```
ubuntu@ubuntu:~/Downloads/lab-cs-cns-00101$ sudo mn --topo=single,4 --controller=remote,port=6655 --switch=ovsk --mac
[sudo] password for ubuntu:
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
```

To change the IP addresses of each container, an Xterm was started for each container and the IP address of the container's interface was changed using the `ifconfig` command:

"Node: h1"

```
root@ubuntu:~/Downloads/lab-cs-cns-00101# ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.2.10 netmask 255.255.255.0 broadcast 192.168.2.255
    ether 00:00:00:00:00:01 txqueuelen 1000 (Ethernet)
    RX packets 173 bytes 14137 (14.1 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 171 bytes 12886 (12.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 20 bytes 1892 (1.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 20 bytes 1892 (1.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@ubuntu:~/Downloads/lab-cs-cns-00101#
```

"Node: h2"

```
root@ubuntu:~/Downloads/lab-cs-cns-00101# ifconfig
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.2.20 netmask 255.255.255.0 broadcast 192.168.2.255
    ether 00:00:00:00:00:02 txqueuelen 1000 (Ethernet)
    RX packets 100 bytes 7744 (7.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 89 bytes 6470 (6.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@ubuntu:~/Downloads/lab-cs-cns-00101#
```

```
"Node: h3"
root@ubuntu:~/Downloads/lab-cs-cns-00101# ifconfig
h3-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.2.30 netmask 255.255.255.0 broadcast 192.168.2.255
    ether 00:00:00:00:00:03 txqueuelen 1000 (Ethernet)
    RX packets 126 bytes 10368 (10.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 89 bytes 6929 (6.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@ubuntu:~/Downloads/lab-cs-cns-00101#
```

```
"Node: h4"
root@ubuntu:~/Downloads/lab-cs-cns-00101# ifconfig
h4-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.4 netmask 255.0.0.0 broadcast 0.0.0.0
    ether 00:00:00:00:00:04 txqueuelen 1000 (Ethernet)
    RX packets 31 bytes 3578 (3.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3 bytes 310 (310.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@ubuntu:~/Downloads/lab-cs-cns-00101# ifconfig h4-eth0 192.168.2.40
root@ubuntu:~/Downloads/lab-cs-cns-00101#
```

New firewall rules were added to the L3 and L2 firewall configurations to container filtering rules per the project instructions:

L3 Firewall (rules 7 – 11 added)

```
ubuntu@ubuntu:~/pox$ cat l3firewall.config
priority,src_mac,dst_mac,src_ip,dst_ip,src_port,dst_port,nw_proto
1,any,any,10.0.0.1,10.0.0.2,1,1,icmp
2,any,any,10.0.0.2,10.0.0.1,1,1,icmp
3,any,any,10.0.0.1,10.0.0.2,any,any,tcp
4,any,any,10.0.0.2,10.0.0.1,any,any,tcp
5,any,any,10.0.0.1,10.0.0.2,any,any,udp
6,00:00:00:00:00:03,any,10.0.0.3,any,any,any,tcp
7,any,any,192.168.2.10,192.168.2.30,1,1,icmp
8,any,any,192.168.2.20,192.168.2.40,1,1,icmp
9,any,any,any,192.168.2.20,any,80,tcp
10,any,any,192.168.2.10,192.168.2.20,any,any,tcp
11,any,any,192.168.2.10,192.168.2.20,any,any,udp
ubuntu@ubuntu:~/pox$
```

L2 Firewall (rule 3 added)

```
ubuntu@ubuntu: ~/pox 101x30
ubuntu@ubuntu:~/pox$ cat l2firewall.config
id,mac_0,mac_1
1,00:00:00:00:00:03,00:00:00:00:00:04
2,00:00:00:00:00:03,00:00:00:00:00:08
3,00:00:00:00:00:02,00:00:00:00:00:04
ubuntu@ubuntu:~/pox$
```

A display of the configuration of the network is shown below:

```
ubuntu@ubuntu: ~/pox 10
ubuntu@ubuntu:~/pox$ sudo ovs-vsctl show
[sudo] password for ubuntu:
b0159eb6-dfd1-4693-b7a2-f90817a738ac
    Bridge "s1"
        Controller "ptcp:6654"
        Controller "tcp:127.0.0.1:6655"
        is_connected: true
        fail_mode: secure
        Port "s1-eth3"
            Interface "s1-eth3"
        Port "s1-eth1"
            Interface "s1-eth1"
        Port "s1-eth4"
            Interface "s1-eth4"
        Port "s1"
            Interface "s1"
            type: internal
        Port "s1-eth2"
            Interface "s1-eth2"
    ovs_version: "2.9.8"
ubuntu@ubuntu:~/pox$
```

Tests of Firewall:

Check blocking ICMP traffic from 192.168.2.10 to 192.168.2.30:

```
ubuntu@ubuntu: ~/Downloads/lab-cs-cns-00101 100x30
containernetwork> h1 ping h3
PING 192.168.2.30 (192.168.2.30) 56(84) bytes of data.
64 bytes from 192.168.2.30: icmp_seq=1 ttl=64 time=20.7 ms
^C
--- 192.168.2.30 ping statistics ---
63 packets transmitted, 1 received, 98% packet loss, time 63446ms
rtt min/avg/max/mdev = 20.733/20.733/20.733/0.000 ms
containernetwork> S
```

One ping went through as expected and then the controller followed the flow rules to block the remaining ICMP packets.

Check blocking ICMP traffic from 192.168.2.20 to 192.168.2.40:

```
ubuntu@ubuntu: ~/Downloads/lab-cs-cns-00101 100x30
containernetwork> h2 ping h4
PING 192.168.2.40 (192.168.2.40) 56(84) bytes of data.
^C
--- 192.168.2.40 ping statistics ---
15 packets transmitted, 0 received, 100% packet loss, time 14332ms
containernetwork>
```

Check blocking HTTP traffic from 192.168.2.20 to any requestor.

To test this, a HTTP server was set up on Node 2:

```
"Node: h2"
root@ubuntu:~/Downloads/lab-cs-cns-00101# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

An HTTP request was sent from the remaining nodes:

```
ubuntu@ubuntu: ~/Downloads/lab-cs-cns-00101 100x30
containernetwork> h1 curl h2
^C
containernetwork> h3 curl h2
^C
containernetwork> h4 curl h2
^C
containernetwork>
```

None of the HTTP requests were successful.

Check MAC address filtering from Node 2 to Node 4.

To perform this test, ARPING was used.

```
ubuntu@ubuntu: ~/Downloads/lab-cs-cns-00101 100x30
containernetwork> h2 arping h4
ARPING 192.168.2.40 from 192.168.2.20 h2-eth0
Unicast reply from 192.168.2.40 [00:00:00:00:00:04] 0.777ms
^CSent 15 probes (1 broadcast(s))
Received 1 response(s)
containernetwork> h1 arping h3
ARPING 192.168.2.30 from 192.168.2.10 h1-eth0
Unicast reply from 192.168.2.30 [00:00:00:00:00:03] 0.814ms
Unicast reply from 192.168.2.30 [00:00:00:00:00:03] 0.755ms
Unicast reply from 192.168.2.30 [00:00:00:00:00:03] 0.561ms
Unicast reply from 192.168.2.30 [00:00:00:00:00:03] 0.579ms
Unicast reply from 192.168.2.30 [00:00:00:00:00:03] 0.542ms
^CSent 5 probes (1 broadcast(s))
Received 5 response(s)
containernetwork>
```

It can be seen that only 1 response is received from Node 4, while testing ARPING from Node 1 to Node 3 produced many responses. This is taken as traffic from Node 2 to Node 4 is blocked based on MAC addresses.

Check blocking of TCP traffic from Node 1 to Node 2:

```
ubuntu@ubuntu: ~/Downloads/lab-cs-cns-00101 100x30
containernetwork> h1 hping3 h2 -V --tcp-timestamp
using h1-eth0, addr: 192.168.2.10, MTU: 1500
HPING 192.168.2.20 (h1-eth0 192.168.2.20): NO FLAGS are set, 40 headers + 0 data bytes
^C
--- 192.168.2.20 hping statistic ---
40 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
containernetwork>
```

As can be seen, all TCP traffic is blocked from Node 1 to Node 2.

Check blocking of UDP traffic from Node 1 to Node 2:

```
ubuntu@ubuntu: ~/Downloads/lab-cs-cns-00101 100x30
containernetwork> h1 hping3 h2 -V -2
using h1-eth0, addr: 192.168.2.10, MTU: 1500
HPING 192.168.2.20 (h1-eth0 192.168.2.20): udp mode set, 28 headers + 0 data bytes
^C
--- 192.168.2.20 hping statistic ---
13 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
containernetwork>
```

As can be seen, all UDP traffic is blocked from Node 1 to Node 2.

V. CONCLUSION

An Open Virtual Switch was set up and 4 containers were connected to it. Firewall rules were set into place to provide security. The firewall was tested against various test conditions. After performing the labs, the project was pretty straightforward. One thing that I did notice is that the firewall would stop working after a time. After restarting the POX process, the firewall would work correctly. I am not sure if this is a bug or proper behavior.